

駝羊的障礙

一隻美洲駝羊想要穿越安地斯高原。它有一張由 $N \times M$ 方格組成的高原地圖。地圖的行從上至下的編號是 0 到 $N - 1$ ，列從左至右的編號是 0 到 $M - 1$ 。地圖上第 i 行、第 j 列 ($0 \leq i < N, 0 \leq j < M$) 的方格以 (i, j) 表示。

駝羊研究了高原的氣候，發現地圖上每一行的所有方格都有相同的**溫度**，而地圖上每一列的所有方格都有相同的**濕度**。駝羊給了您兩個長度分別為 N 和 M 的整數陣列 T 和 H 。這裡 $T[i]$ ($0 \leq i < N$) 表示 i 行中的方格的溫度，而 $H[j]$ ($0 \leq j < M$) 表示 j 列中的方格的濕度。

駝羊也研究過高原上的植物群，並注意到當且僅當 (i, j) 的溫度大於濕度時，該方格上才**沒有植被**，形式上 $T[i] > H[j]$ 。

駝羊只能沿著**有效路徑**穿越高原。有效路徑是滿足下列條件的不同方格序列：

- 路徑中的每對連續方格都有一條公共的邊。
- 路徑中的所有方格都沒有植被。

您的任務是回答 Q 個問題。每個問題都會給您四個整數： L, R, S 和 D 。您必須判斷是否存在有效的路徑，使得：

- 該路徑從 $(0, S)$ 方格開始，到 $(0, D)$ 方格結束。
- 路徑中的所有方格都位於 L 至 R 列（包括首尾兩列）。

保證 $(0, S)$ 和 $(0, D)$ 都沒有植被。

編程細節

您應該編寫的第一個子程序是：

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T ：一個長度為 N 的陣列，指定每一行的溫度。
- H ：一個長度為 M 的陣列，指定每列的濕度。
- 在呼叫 `can_reach` 之前，每個測試個案都會呼叫一次這個程序。

您應該編寫的第二個子程序是：

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : 描述問題的整數。
- 每個測試樣例都會呼叫此子程序 Q 次。

當且僅當存在一條從 $(0, S)$ 到 $(0, D)$ 的有效路徑時，此程序才會返回 `true`，這樣路徑中的所有儲存格都位於 L 到 R 之間。

約束

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ 對每個 i 其中 $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ 對每個 j 其中 $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- $(0, S)$ 和 $(0, D)$ 兩個方格都沒有被植。

子任務

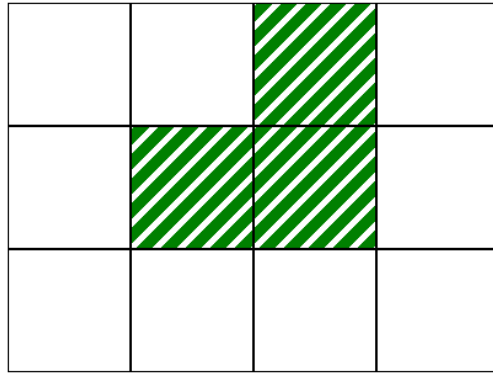
子任務	得分	附加限制條件
1	10	$L = 0, R = M - 1$ 對每個問題。 $N = 1$.
2	14	$L = 0, R = M - 1$ 對每個問題。 $T[i - 1] \leq T[i]$ 對每個 i 使得 $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ 對每個問題。 $N = 3$ 且 $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ 對每個問題。 $Q \leq 10$.
5	25	$L = 0, R = M - 1$ 對每個問題。
6	17	無額外限制。

範例

考慮以下呼叫：

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

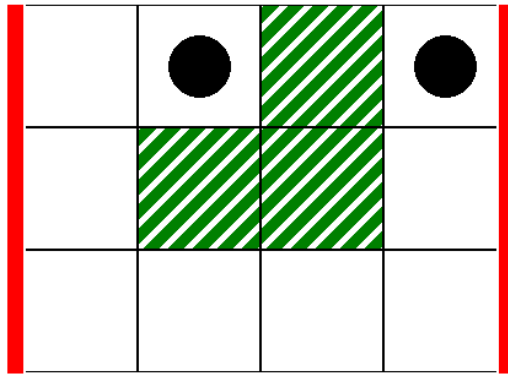
這與下圖中的地圖相符，白色的方格沒有植被：



作為第一個問題，請考慮以下的呼叫：

```
can_reach(0, 3, 1, 3)
```

這與下圖中的情況相對應，粗垂直線表示從 $L = 0$ 到 $R = 3$ 的列範圍，黑色圓盤表示開始和結束的方格：



在這種情況下，駝羊可以透過下列有效路徑從 $(0,1)$ 到 $(0,3)$ 方格：

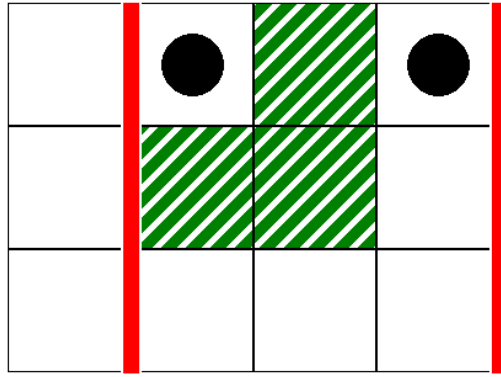
$(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)$

因此，此呼叫應該返回 `true`。

作為第二個問題，請考慮以下的呼叫：

```
can_reach(1, 3, 1, 3)
```

這與下圖的情況相符：



在這種情況下，從 $(0,1)$ 方格到 $(0,3)$ 方格之間沒有有效的路徑，使得路徑中的所有方格都位於 1 到 3 之間的列中。因此，此呼叫應返回 `false`。

樣例評分程式

輸入格式：

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

這裡， $L[k], R[k], S[k]$ 和 $D[k]$ ($0 \leq k < Q$) 指定了每次呼叫 `can_reach` 的參數。

輸出格式：

```
A[0]
A[1]
...
A[Q-1]
```

這裡， $A[k]$ ($0 \leq k < Q$) 是 1，如果呼叫 `can_reach(L[k], R[k], S[k], D[k])` 返回 `true`，否則為 0。